Front End Interview Questions

HTML5 • CSS3 • JavaScript • React • Angular

189 commonly asked front-end developer interview questions with answers

Contents

HTML5 — 37 questions

CSS3 — 32 questions

JavaScript — 76 questions

React — 24 questions

Angular — 20 questions

Each section below covers the topic's most frequently asked interview questions, including the ones most repeated across interview-prep lists, plus a set of additional commonly asked questions to round out your preparation.

HTML5 (37 questions)

1. Why do you need a DOCTYPE?

The DOCTYPE declaration tells the browser which version of HTML the page is written in, so it renders the page in standards mode instead of quirks mode. Without it, older rendering behaviors kick in and layout/CSS can behave inconsistently across browsers.

2. What are data-* attributes used for?

data-* attributes let you store custom data directly on HTML elements without extra properties or JavaScript hacks. They're accessed via the dataset property (e.g., element.dataset.userId) and are commonly used to pass information from markup to JavaScript.

3. How can you generate a public key in HTML?

HTML5 previously supported the <keygen> element to generate a public/private key pair for client certificate authentication. It has since been deprecated and removed from modern browsers and HTML specs in favor of the Web Crypto API.

4. How do you change the direction of HTML text?

Use the dir attribute on an element, e.g., <p dir="rtl">, or the CSS direction property. dir="ltr" is left-to-right (default), dir="rtl" is right-to-left, useful for languages like Arabic or Hebrew.

5. How can you highlight text in HTML?

Use the <mark> element, e.g., <mark>highlighted text</mark>, which renders with a yellow background by default and can be restyled with CSS.

6. Can you apply CSS to a part of an HTML document only?

Yes — using scoped selectors (class/ID), inline styles, or wrapping the section in a container and scoping styles to that container. The now-removed <style scoped> attribute attempted this natively; today it's done via CSS Modules, Shadow DOM, or specific selectors.

7. Will a browser make an HTTP request for the following cases: <img src="">, hidden images, or elements with no src?

An <img> with an empty src (src="") often triggers a request to the current page URL in many browsers — a common bug. Elements with display:none still load their resources (like images) because the DOM node exists; only removing the src/attribute prevents the request.

8. Which resource would be downloaded first: CSS or JS?

By default, CSS in the <head> blocks rendering and is fetched with high priority; synchronous <script> tags block HTML parsing and are fetched as soon as they're encountered, but the browser generally prioritizes render-blocking CSS. Using async/defer on scripts changes load/execution order relative to parsing.

9. What is an optional tag in HTML?

Some HTML tags can be omitted and the browser will still infer their presence per the parsing spec — e.g., <html>, <head>, <body>, <tbody>, and closing tags like , </p>, </tr> in certain contexts. Omitting them is legal but not best practice for readability.

10. What are the differences between div and span?

 is a block-level element used to group larger sections of content (takes full width, starts on a new line). is an inline element used to wrap small pieces of text or inline content without breaking the flow.

11. How would you differentiate between div, section, and article?

 is a generic, non-semantic container. <section> groups thematically related content, typically with a heading. <article> represents a self-contained, independently distributable piece of content (like a blog post or news story) that would still make sense on its own.

12. How would you select SVG or canvas for your site?

Use SVG for resolution-independent vector graphics, icons, and charts that need to scale and be styled/animated via CSS, and where DOM access to individual shapes matters (accessibility, event handling). Use canvas for pixel-based, high-performance drawing like games, image manipulation, or large real-time visualizations.

13. How to serve HTML in multiple languages?

Set the lang attribute on <html> (e.g., lang="en"), use hreflang links for alternate language versions, and serve localized content either via separate URLs per locale, server-side content negotiation (Accept-Language header), or client-side i18n libraries.

14. Explain standard mode and quirks mode.

Standards mode renders a page according to the W3C/HTML specifications, triggered by a valid DOCTYPE. Quirks mode emulates old, non-standard browser behavior (from pre-CSS-standard era) for backward compatibility, typically triggered by a missing or malformed DOCTYPE.

15. What is a semantic tag?

A semantic tag clearly describes its meaning/content to both the browser and developer, e.g., <header>, <nav>, <main>, <article>, <footer>, unlike generic tags like or . They improve accessibility, SEO, and code readability.

16. What are the different Drag and Drop events?

Key events include dragstart, drag, dragenter, dragover, dragleave, drop, and dragend. dragstart fires on the source element, dragover/drop fire on the target (dragover must call preventDefault to allow dropping), and dragend fires when the operation completes.

17. What is HTML5 Web Storage?

Web Storage is a browser API for storing key-value data on the client, larger and simpler than cookies, and not sent with every HTTP request. It includes localStorage and sessionStorage, both accessed via a similar API (setItem, getItem, removeItem).

18. What are the different types of Web Storage provided by HTML5?

localStorage persists data with no expiration date across browser sessions. sessionStorage persists only for the duration of the page session (tab) and is cleared when the tab closes.

19. What is the need for introducing Local Storage in HTML5?

Cookies are limited in size (~4KB), sent on every HTTP request (adding overhead), and awkward to manage. localStorage offers much larger storage (~5-10MB), stays purely client-side, and provides a simple synchronous API for persisting data like preferences or cached data.

20. Explain the differences between localStorage and sessionStorage objects.

localStorage data persists indefinitely (until explicitly cleared) and is shared across all tabs/windows of the same origin. sessionStorage data is scoped to a single tab and is cleared when that tab is closed, even if the same origin is open in another tab.

21. What is the concept of Application Cache in HTML5? What are its advantages?

AppCache let developers specify files to be cached via a manifest file so a web app could work offline. Advantages included offline browsing, faster loading from cache, and reduced server load — but it's deprecated in favor of Service Workers due to its many caching pitfalls.

22. What is a Manifest file?

In the context of AppCache, it's a text file listing resources the browser should cache for offline use, referenced via the manifest attribute on <html>. In modern web apps, a Web App Manifest (JSON) instead describes app metadata (icons, name, start URL) for PWAs.

23. Explain HTML5 Application cache vs. regular HTML browser cache.

The regular browser (HTTP) cache stores individual resources based on cache headers and may evict them anytime based on heuristics. AppCache explicitly declared a whole set of resources for guaranteed offline availability via a manifest — but it's now deprecated in favor of the more flexible, programmable Service Worker Cache API.

24. What is a Web Worker?

A Web Worker runs JavaScript in a background thread separate from the main UI thread, allowing CPU-intensive tasks to run without blocking page rendering or user interaction.

25. How does a Web Worker work?

You create a worker with new Worker('script.js'); it runs in an isolated global scope with no DOM access. Communication with the main thread happens via postMessage() and the onmessage event handler, passing serializable data (or transferable objects) between threads.

26. What are the new attributes provided in HTML5 for <form>?

New attributes include autocomplete, autofocus, form, formaction, formenctype, formmethod, formnovalidate, formtarget, novalidate, and required, giving finer control over validation and submission behavior.

27. What is the output element in HTML5?

<output> represents the result of a calculation or user action, often used with JavaScript and forms — e.g., <output name="result">, updated dynamically to show computed values.

28. What are the new attributes provided in HTML5 for the <input> element?

New attributes include type values like email, url, number, range, date, color, search; plus attributes like placeholder, required, pattern, min, max, step, autofocus, and list (for datalist binding).

29. What is the major difference between Transitional and Strict DOCTYPE?

Strict DOCTYPE enforces standards-compliant markup and disallows deprecated/presentational elements and attributes (like <font> or align). Transitional DOCTYPE is more lenient, allowing deprecated presentational markup for backward compatibility with older pages.

30. What is the Audio tag in HTML5?

<audio> embeds sound content directly in a page without plugins, e.g., <audio controls src="song.mp3"></audio>, supporting multiple source formats via nested <source> elements.

31. What are HTML5 audio tag attributes?

Common attributes: src, controls, autoplay, loop, muted, preload (none/metadata/auto), and multiple <source> children for format fallback (mp3, ogg, wav).

32. What is the use of the <fieldset> tag in HTML5?

<fieldset> groups related form controls together visually and semantically, often paired with <legend> to provide a caption for the group — improving accessibility for screen readers.

33. What are the HTML tags which get deprecated in HTML5?

Deprecated/removed tags include <font>, , <big>, <strike>, <tt>, <frame>, <frameset>, <acronym>, and <applet>, generally replaced by CSS styling or semantic alternatives.

34. What is a meter tag? What is the difference between a progress tag and a meter tag?

<meter> represents a scalar measurement within a known range (like disk usage or a rating), while <progress> represents the completion progress of a task (like a file upload). Meter is for static/gauge-like values; progress is for an ongoing, changing task.

35. Why do we need HTML5 Server-Sent Events?

Server-Sent Events (SSE) allow a server to push real-time updates to the browser over a single, long-lived HTTP connection without the client repeatedly polling — useful for live feeds, notifications, or stock tickers, and simpler than WebSockets when you only need one-way server-to-client communication.

36. Explain how to receive Server-Sent Event notifications.

The client creates an EventSource object pointed at a server endpoint: const source = new EventSource('/events'); source.onmessage = e => console.log(e.data);. The browser automatically maintains and reconnects the connection if it drops.

37. Explain a Server-Side code example for SSE.

The server keeps the HTTP response open and sets Content-Type: text/event-stream, sending messages formatted as 'data: <message>\n\n'. For example, in Node/Express you'd write res.write(`data: ${payload}\n\n`) periodically without ending the response.

CSS3 (32 questions)

1. What does float do?

float removes an element from normal document flow and shifts it to the left or right, allowing text/inline content to wrap around it. It was traditionally used for layouts before Flexbox/Grid, and still commonly used for wrapping text around images.

2. How can you clear the sides of a floating element?

Use the clear property (clear: left, right, or both) on a following element, or apply a clearfix hack (a pseudo-element with content:'' and clear:both) to the parent container to prevent collapsing height around floated children.

3. Are CSS rule names (selectors) case sensitive?

HTML tag selectors and property names are case-insensitive, but class and ID selectors are case-sensitive because HTML class/id attribute values themselves are case-sensitive in most contexts (especially XHTML/XML).

4. Why do CSS selectors mixed up with cases not apply the styles?

Because class and ID values are case-sensitive, a selector like .myClass won't match an element with class="myclass" — the mismatch in casing means the browser treats them as different identifiers entirely.

5. Does margin-top or margin-bottom have an effect on inline elements?

No. Vertical margins (margin-top, margin-bottom) have no effect on inline (non-replaced) elements; they don't affect the line box height or layout at all.

6. Does padding-top or padding-bottom have an effect on inline elements?

Vertical padding is applied visually (background/border expand) on inline elements, but it doesn't affect the line height or push surrounding lines apart — so it can visually overlap adjacent content.

7. Does padding-left/right or margin-left/right have an effect on inline elements?

Yes — horizontal padding and margin on inline elements do affect layout, adding actual visible space and pushing neighboring inline content horizontally within the line.

8. If a <p> element has font-size:10rem, will the text be responsive when the user resizes the browser window?

rem units are relative to the root (<html>) element's font-size, not the viewport, so simply resizing the browser window won't change the text size unless the root font-size itself is set responsively (e.g., via media queries or viewport units like vw).

9. The pseudo-class :checked will select inputs with type radio or checkbox, but not <option> elements — true or false?

This is false as stated for older specs but true in modern CSS — :checked does apply to selected <option> elements as well as checked radio/checkbox inputs in current browsers.

10. In an HTML document, the pseudo-class :root always refers to the <html> element — true or false?

True. :root refers to the highest-level parent in the document tree, which in an HTML document is always the <html> element (equivalent to selecting html but with higher specificity).

11. The translate() function can move the position of an element on the z-axis — true or false?

False. The 2D translate() function only moves along the x and y axes. To move along the z-axis you need translate3d() or translateZ(), which requires a 3D rendering context (perspective).

12. Which one would you prefer among px, em, % or pt and why?

It depends on context: px gives precise, fixed control (good for borders); em/rem scale relative to parent/root font-size, making layouts more accessible and responsive to user font-size preferences; % is useful for fluid widths; pt is mainly for print. Modern practice favors rem for typography and % / flexible units for responsive layout.

13. How do absolute, relative, fixed, and static position differ?

static is the default flow position (no offset properties apply). relative positions an element offset from its normal position while still occupying its original space. absolute removes it from flow and positions it relative to the nearest positioned ancestor. fixed positions it relative to the viewport, staying in place during scroll.

14. What are the differences between visibility:hidden and display:none?

display:none removes the element entirely from the layout (no space reserved, not rendered). visibility:hidden hides the element visually but it still occupies its layout space and remains part of the accessibility tree in some respects (though not exposed to assistive tech in most browsers).

15. What are the differences between inline, block and inline-block?

block elements start on a new line and take the full available width. inline elements flow within text and don't accept width/height or vertical margins. inline-block flows inline like text but accepts width, height, and full margin/padding like a block.

16. What are the properties related to the box model?

content, padding, border, and margin — together they determine an element's total rendered size. The box-sizing property (content-box vs border-box) determines whether width/height include padding and border.

17. Does overflow:hidden create a new block formatting context?

Yes. Setting overflow to any value other than visible (including hidden) establishes a new Block Formatting Context, which contains floats and prevents margin collapsing with children.

18. How could you apply CSS rules specific to a media (device/screen size)?

Use @media queries, e.g., @media (max-width: 768px) { ... }, to apply styles conditionally based on viewport width, orientation, resolution, or device type.

19. What is the use of 'only' in media queries?

The 'only' keyword prevents older browsers that don't support media queries from applying the styles by hiding the media type from them, e.g., @media only screen and (max-width: 600px).

20. Does the 'screen' keyword apply to the device's physical screen or the browser's viewport?

It refers to the browser's rendering viewport (used for regular color displays), not necessarily the full physical device screen — as opposed to the print media type used for printed output.

21. What are some pseudo classes you have used?

:hover, :focus, :active, :first-child, :last-child, :nth-child(), :not(), :checked, :disabled, and :visited are common examples used for interactive states and structural targeting.

22. How do you align a <p> center-center inside a div?

Using Flexbox: set the parent to display:flex; justify-content:center; align-items:center;. Alternatively use CSS Grid with place-items:center, or absolute positioning with transform:translate(-50%,-50%).

23. How do you optimize CSS selectors?

Keep selectors short and specific (avoid deep descendant chains), prefer classes over complex attribute/tag selectors, avoid the universal selector in hot paths, minimize use of !important, and avoid overly generic rules matched against many elements.

24. How can you load CSS resources conditionally?

Use media attributes on <link> tags (e.g., media="print"), JavaScript to dynamically inject <link> elements based on conditions, or CSS @import with media conditions — though @import can hurt performance due to blocking, sequential downloads.

25. Why would you use sprites?

CSS sprites combine multiple small images into a single file, reducing the number of HTTP requests and improving load performance — you display the right portion using background-position. Less relevant now with HTTP/2 multiplexing and modern image techniques.

26. What is specificity? How do you calculate specificity?

Specificity determines which CSS rule wins when multiple rules target the same element. It's calculated as a weighted score: inline styles > IDs > classes/attributes/pseudo-classes > element/pseudo-element selectors, often written as (a,b,c,d) tuples.

27. What is Shadow DOM?

Shadow DOM is a browser technology that lets you attach a hidden, encapsulated DOM tree to an element, scoping its styles and markup away from the main document — the foundation of native Web Components.

28. What do you know about transitions?

CSS transitions animate property changes smoothly over a specified duration, using properties like transition-property, transition-duration, transition-timing-function, and transition-delay, triggered by state changes like :hover or class toggles.

29. What are the different CSS filters you can use?

Filters include blur(), brightness(), contrast(), grayscale(), hue-rotate(), invert(), opacity(), saturate(), sepia(), and drop-shadow(), applied via the filter property to alter an element's rendering.

30. What are the reasons to use a preprocessor?

Preprocessors like Sass/Less add variables, nesting, mixins, functions, and partials/imports that plain CSS historically lacked, improving maintainability and reducing repetition — though native CSS now supports variables and nesting too.

31. What is the difference between Flexbox and CSS Grid?

Flexbox is a one-dimensional layout system (row or column) best for distributing space among items in a single direction. CSS Grid is two-dimensional, letting you control rows and columns simultaneously for complex page layouts.

32. What is the CSS box-sizing property and why is border-box preferred?

box-sizing:content-box (default) makes width/height apply only to content, with padding/border added on top, causing unexpected sizing. box-sizing:border-box includes padding and border within the specified width/height, making sizing far more predictable and is widely used as a reset default.

JavaScript (76 questions)

1. What is JavaScript?

JavaScript is a high-level, interpreted, dynamically-typed programming language primarily used to add interactivity and logic to web pages, running natively in browsers and, via Node.js, on servers.

2. What are the data types in JavaScript?

Primitive types: string, number, boolean, undefined, null, symbol, and bigint. Non-primitive: object (including arrays and functions).

3. What is the difference between undefined and null?

undefined means a variable has been declared but not assigned a value (or a property/function doesn't exist). null is an explicit assignment representing 'no value,' intentionally set by the developer.

4. What is the difference between == and ===?

== (loose equality) compares values after performing type coercion if types differ. === (strict equality) compares both value and type without coercion, making it generally the safer, more predictable choice.

5. What is the difference between var, let, and const?

var is function-scoped and hoisted with an 'undefined' initial value; it can be redeclared. let is block-scoped, hoisted but in a temporal dead zone, and can be reassigned but not redeclared in the same scope. const is block-scoped like let but cannot be reassigned after initialization (though object/array contents it references can still be mutated).

6. What is hoisting in JavaScript?

Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope during compilation. var declarations are hoisted and initialized as undefined; let/const are hoisted but remain uninitialized in the 'temporal dead zone' until their actual declaration line executes.

7. What is the 'this' keyword in JavaScript?

'this' refers to the execution context of a function — the object it's called on. Its value depends on how a function is invoked (method call, standalone call, constructor call, or explicit binding via call/apply/bind), and in arrow functions it's lexically inherited from the enclosing scope.

8. What is event bubbling in JavaScript?

Event bubbling is when an event triggered on a nested element propagates upward through its ancestors in the DOM tree, triggering their handlers too, unless stopped with event.stopPropagation().

9. What is a closure in JavaScript?

A closure is a function that retains access to variables from its enclosing lexical scope even after that outer function has finished executing. Closures are commonly used for data privacy, memoization, and creating factory functions.

10. What is a higher-order function in JavaScript?

A higher-order function is one that takes another function as an argument, returns a function, or both — examples include map(), filter(), reduce(), and custom function factories.

11. What is a callback function in JavaScript?

A callback is a function passed as an argument to another function, to be executed after some operation completes — commonly used for asynchronous operations like API calls, timers, or event handlers.

12. What is a pure function in JavaScript?

A pure function always returns the same output for the same input and produces no side effects (doesn't modify external state, doesn't perform I/O), making it predictable and easy to test.

13. What is the difference between function declaration and function expression?

A function declaration (function foo(){}) is hoisted entirely and can be called before its definition in code. A function expression (const foo = function(){}) is only hoisted as a variable (not the assignment), so it can't be called before the line where it's defined.

14. What is a recursive function in JavaScript?

A recursive function calls itself to solve smaller instances of a problem until it reaches a base case that stops the recursion, commonly used for tasks like tree traversal, factorial calculation, or divide-and-conquer algorithms.

15. What is a generator function in JavaScript?

A generator function, defined with function*, can pause and resume execution using the yield keyword, returning an iterator that produces a sequence of values on demand rather than all at once.

16. What is the difference between .call() and .apply()?

Both invoke a function with a specified 'this' value, but call() takes arguments individually (fn.call(obj, a, b)), while apply() takes them as a single array (fn.apply(obj, [a, b])).

17. What is the difference between .bind() and .call()?

call() invokes the function immediately with a given 'this'. bind() instead returns a new function with 'this' permanently bound, which can be invoked later rather than immediately.

18. What is the arguments object in JavaScript?

arguments is an array-like object available inside regular (non-arrow) functions containing all arguments passed to the function, regardless of the named parameters — not available in arrow functions.

19. What is a rest parameter in JavaScript?

The rest parameter (...args) collects all remaining arguments passed to a function into a real array, allowing functions to accept a variable number of arguments cleanly, e.g., function sum(...nums){}.

20. What is an object in JavaScript?

An object is a collection of key-value pairs (properties), where values can be primitives, other objects, or functions (methods) — the core building block for structuring data in JavaScript.

21. What is the difference between an object and an array in JavaScript?

An array is an ordered list of values indexed numerically, optimized for sequential data. An object is an unordered collection of key-value pairs, better suited for representing named properties/entities.

22. What is prototypal inheritance in JavaScript?

Every JavaScript object has an internal link (__proto__) to another object, its prototype, from which it inherits properties and methods. If a property isn't found on the object itself, JavaScript looks up the prototype chain until it finds it or reaches null.

23. What is the Object.create() method in JavaScript?

Object.create(proto) creates a new object with the specified object as its prototype, allowing explicit control over prototypal inheritance without using constructor functions or classes.

24. What is the 'new' keyword in JavaScript?

'new' creates a new empty object, sets its prototype to the constructor function's prototype, executes the constructor with 'this' bound to the new object, and returns that object (unless the constructor explicitly returns another object).

25. What is a constructor function in JavaScript?

A constructor function is a regular function intended to be called with 'new' to create and initialize new object instances, conventionally named with a capital letter, e.g., function Person(name){ this.name = name; }.

26. What is an array in JavaScript?

An array is an ordered, index-based collection of values (of any type), with built-in methods for iteration, transformation, and manipulation.

27. What is the difference between .forEach() and .map()?

forEach() executes a callback for each array element but returns undefined (used for side effects). map() also executes a callback for each element but returns a brand-new array containing the transformed values.

28. What is the difference between .slice() and .splice()?

slice(start, end) returns a shallow copy of a portion of the array without modifying the original. splice(start, deleteCount, ...items) mutates the original array by removing/replacing/inserting elements in place.

29. What is the difference between .shift() and .pop()?

shift() removes and returns the first element of an array, shifting remaining elements down. pop() removes and returns the last element, which is generally more performant since no re-indexing is needed.

30. What is the Array.prototype.filter() method in JavaScript?

filter() creates a new array containing only the elements that pass a provided test function (predicate), leaving the original array unmodified.

31. What is the Array.prototype.reduce() method in JavaScript?

reduce() executes a reducer function on each array element, accumulating a single output value (sum, object, flattened array, etc.) by carrying an accumulator through each iteration.

32. What are template literals in JavaScript?

Template literals use backticks (`) and allow embedded expressions via ${expr}, along with native multi-line strings, e.g., `Hello, ${name}!`.

33. What are default parameters in JavaScript?

Default parameters let you specify a fallback value for a function argument if none (or undefined) is passed, e.g., function greet(name = 'Guest'){}.

34. What are destructuring assignments in JavaScript?

Destructuring lets you unpack values from arrays or properties from objects into distinct variables in a concise syntax, e.g., const {name, age} = user; or const [a, b] = arr;.

35. What is the spread operator in JavaScript?

The spread operator (...) expands an iterable (array, string) or object's own enumerable properties into individual elements — used for copying arrays/objects, merging, or passing array elements as function arguments.

36. What is the class keyword in JavaScript?

class provides syntactic sugar over JavaScript's existing prototypal inheritance, offering a cleaner, more familiar syntax for defining constructors, methods, and inheritance (extends) — but it doesn't introduce classical (class-based) inheritance under the hood.

37. What is the super keyword in JavaScript?

super is used inside a derived class to call the parent class's constructor (super(args)) or to access parent class methods (super.methodName()) that have been overridden.

38. What is the Symbol data type in JavaScript?

Symbol is a primitive type that creates unique, immutable identifiers, often used as non-colliding object property keys, especially for defining well-known meta-behaviors (like Symbol.iterator).

39. What are promises in JavaScript?

A Promise is an object representing the eventual completion (or failure) of an asynchronous operation, with three states: pending, fulfilled, or rejected — allowing chaining via .then()/.catch() instead of nested callbacks.

40. What is the difference between a promise and a callback?

Callbacks are plain functions passed to be invoked later, which can lead to deeply nested 'callback hell' for sequential async operations. Promises provide a structured, chainable API with built-in error propagation, composability (Promise.all, etc.), and better readability.

41. What is the .then() method in promises?

then() attaches callbacks to a promise to handle its fulfilled (and optionally rejected) result, returning a new promise, enabling chaining of asynchronous steps.

42. What is the .catch() method in promises?

catch() attaches a callback specifically for handling a promise's rejection (error), equivalent to .then(null, onRejected), and it also catches errors thrown earlier in the chain.

43. What is the .finally() method in promises?

finally() registers a callback that runs once a promise settles, regardless of whether it was fulfilled or rejected — commonly used for cleanup tasks like hiding a loading spinner.

44. What is the Promise.all() method in JavaScript?

Promise.all() takes an array of promises and returns a single promise that resolves when all input promises resolve (with an array of their results), or rejects immediately if any one of them rejects.

45. What is the Promise.race() method in JavaScript?

Promise.race() returns a promise that settles as soon as the first of the input promises settles (whether fulfilled or rejected), useful for timeout patterns.

46. What is the Promise.allSettled() method in JavaScript?

Promise.allSettled() waits for all promises to settle (regardless of success or failure) and returns an array of result objects describing each outcome (status: 'fulfilled' or 'rejected'), unlike Promise.all() which short-circuits on the first rejection.

47. What is the async keyword in JavaScript?

async before a function declaration marks it as asynchronous, meaning it always returns a Promise and allows the use of the await keyword inside it.

48. What is the await keyword in JavaScript?

await pauses execution of an async function until a Promise settles, then resumes with the resolved value (or throws if rejected), making asynchronous code read like synchronous code.

49. What is the difference between async/await and promises?

async/await is syntactic sugar built on top of Promises, offering more readable, sequential-looking code instead of chained .then() calls — under the hood, it still uses the same Promise mechanics.

50. How do you handle errors in async/await?

Wrap the awaited code in a try/catch block; any rejection from an awaited promise throws inside the try block and is caught in the corresponding catch block.

51. What is the try/catch block in JavaScript?

try/catch lets you attempt code that might throw an error in the try block, and gracefully handle any thrown exception in the catch block, optionally followed by a finally block for cleanup.

52. How do you use async/await with Promise.all()?

Await the Promise.all() call directly: const results = await Promise.all([p1, p2, p3]);, which runs the promises concurrently and resolves once all have completed, rather than awaiting each one sequentially.

53. What is the Document Object Model (DOM) in JavaScript?

The DOM is a programming interface/tree representation of an HTML document, where each element, attribute, and piece of text is a node that JavaScript can read and manipulate.

54. What is the difference between innerHTML and textContent?

innerHTML gets/sets the HTML markup inside an element (parsing any tags), which can introduce security risks (XSS) if used with untrusted input. textContent gets/sets only the raw text content, ignoring/escaping any HTML tags, and is safer and generally faster.

55. What is the querySelector() method in JavaScript?

querySelector() returns the first element matching a given CSS selector, e.g., document.querySelector('.item'); querySelectorAll() returns all matching elements as a static NodeList.

56. What is the addEventListener() method in JavaScript?

addEventListener(type, handler, options) attaches an event handler to an element without overwriting existing handlers, and supports options like capture phase or {once: true}.

57. What is event delegation in JavaScript?

Event delegation attaches a single event listener to a parent element to handle events for its (current and future) child elements, using event bubbling and event.target to identify which child triggered it — more efficient than binding listeners to many individual children.

58. What are browser APIs in JavaScript?

Browser APIs are built-in interfaces provided by the browser (not the JS language itself) for interacting with the browser environment — e.g., DOM API, Fetch API, Geolocation, localStorage, Web Workers, and the History API.

59. What is the localStorage API in JavaScript?

localStorage provides key-value storage in the browser that persists across sessions with no expiration, accessed via localStorage.setItem(key, value) and localStorage.getItem(key).

60. What is the sessionStorage API in JavaScript?

sessionStorage stores key-value data scoped to a single browser tab/session, cleared when that tab closes, using the same API shape as localStorage.

61. What is the window object in JavaScript?

window is the global object in browser JavaScript, representing the browser window/tab, providing access to the DOM (via window.document), timers, location, history, and global variables/functions.

62. What is the setTimeout() method in JavaScript?

setTimeout(callback, delay) schedules a function to run once after a specified delay (in milliseconds), returning a timer ID that can be used to cancel it with clearTimeout().

63. What is the event loop in JavaScript?

The event loop is the mechanism that allows JavaScript's single-threaded runtime to handle asynchronous operations: it continuously checks the call stack and, when empty, pulls the next task from the callback/microtask queues, enabling non-blocking behavior.

64. What is the requestAnimationFrame() method in JavaScript?

requestAnimationFrame(callback) schedules a function to run before the browser's next repaint, synchronized with the display refresh rate, producing smoother animations than setTimeout/setInterval.

65. What is memoization in JavaScript?

Memoization is an optimization technique that caches the results of expensive function calls based on their input arguments, returning the cached result for repeated calls with the same inputs instead of recomputing.

66. What is lazy loading in JavaScript?

Lazy loading defers loading of resources (images, components, modules) until they're actually needed — e.g., loading images as they scroll into view, or dynamically importing JS modules — improving initial load performance.

67. What is tree shaking in JavaScript?

Tree shaking is a build-time optimization (used by bundlers like Webpack/Rollup) that removes unused ('dead') code from the final bundle, relying on ES module static import/export structure to determine what's actually used.

68. What is Cross-Site Scripting (XSS) in JavaScript?

XSS is a security vulnerability where an attacker injects malicious scripts into a web page (often via unsanitized user input rendered as HTML), which then execute in other users' browsers, potentially stealing data or hijacking sessions.

69. What is Cross-Site Request Forgery (CSRF)?

CSRF tricks an authenticated user's browser into unknowingly submitting a malicious request to a site they're logged into, exploiting the browser's automatic inclusion of cookies — mitigated via CSRF tokens, SameSite cookies, and origin checks.

70. What is the Same-Origin Policy in JavaScript?

The Same-Origin Policy restricts scripts on one origin (protocol + domain + port) from accessing data on a different origin, preventing malicious sites from reading sensitive data from other sites a user is logged into.

71. What is Content Security Policy (CSP) in JavaScript?

CSP is an HTTP response header that lets a site specify which sources of scripts, styles, images, etc. are allowed to load/execute, significantly reducing the risk and impact of XSS attacks.

72. What is the difference between encryption and hashing?

Encryption is reversible: encrypted data can be decrypted back to its original form with the correct key. Hashing is one-way: it transforms data into a fixed-size digest that cannot be reversed, commonly used

for verifying integrity or storing passwords.

73. What is the difference between null and NaN?

null represents the intentional absence of any object value. NaN ('Not a Number') is a special numeric value representing the result of an invalid or undefined mathematical operation, and notably NaN !== NaN.

74. What is debouncing and throttling?

Debouncing delays executing a function until a certain time has passed since the last call (useful for search-as-you-type). Throttling ensures a function executes at most once per specified time interval, regardless of how many times it's triggered (useful for scroll/resize handlers).

75. What is the difference between synchronous and asynchronous code?

Synchronous code executes sequentially, blocking further execution until the current operation finishes. Asynchronous code allows operations (like network requests or timers) to run in the background, letting the rest of the program continue without waiting.

76. What are WeakMap and WeakSet in JavaScript?

WeakMap and WeakSet store object keys/values 'weakly', meaning entries don't prevent garbage collection if there are no other references to the object — useful for associating metadata with objects without causing memory leaks.

React (24 questions)

1. What are the limitations of React?

React is only a view (UI) library, not a full framework — it doesn't include built-in routing, state management, or HTTP handling, requiring additional libraries (React Router, Redux/Zustand, etc.). It also has a learning curve around JSX, hooks rules, and can lead to fast-changing tooling/ecosystem churn.

2. What are the recommended ways for static type checking in React?

TypeScript is the most widely recommended option, offering full static typing for props, state, and hooks. PropTypes is a lighter-weight, runtime-only alternative for smaller projects or plain JavaScript codebases.

3. What is the use of the react-dom package?

react-dom provides DOM-specific methods that let React render components into the actual browser DOM, including ReactDOM.createRoot().render() for mounting apps and createPortal() for rendering outside the normal DOM hierarchy.

4. What is ReactDOMServer?

ReactDOMServer is an API for rendering React components to static HTML strings on the server (renderToString, renderToPipeableStream), enabling server-side rendering (SSR) for faster initial paint and better SEO.

5. How do you use innerHTML in React?

React discourages direct DOM manipulation, but you can render raw HTML using the dangerouslySetInnerHTML prop: <div dangerouslySetInnerHTML={{__html: htmlString}} />, named deliberately to highlight the XSS risk of unsanitized input.

6. How do you apply styles in React?

Common approaches include inline styles via a style object (style={{color:'red'}}), CSS/Sass stylesheets imported into components, CSS Modules for scoped class names, or CSS-in-JS libraries like styled-components/Emotion.

7. How are events different in React?

React uses SyntheticEvents, a cross-browser wrapper around native DOM events, providing consistent behavior across browsers. Event handlers are attached via camelCase props (onClick) directly in JSX rather than via addEventListener, and React handles delegation internally.

8. What is the impact of using indexes as keys?

Using array indexes as keys can cause bugs when list items are reordered, inserted, or removed — React may misidentify which DOM node maps to which data item, causing incorrect re-renders, lost component state, or UI glitches. Stable, unique IDs should be used instead.

9. How do you conditionally render components?

Common patterns include ternary operators ({condition ? <A/> : <B/>}), logical AND ({condition && <A/>}), early returns inside the component function, or separate variables holding JSX assigned conditionally before the return statement.

10. Why do we need to be careful when spreading props on DOM elements?

Spreading arbitrary props onto a native DOM element (e.g., <div {...props}>) can pass through invalid/unknown HTML attributes, causing React warnings or unintended attributes in the rendered markup — it's safer to explicitly destructure and pass only valid DOM attributes.

11. How do you memoize a component?

Wrap a component with React.memo() to prevent re-renders when props haven't changed (shallow comparison). Use useMemo() to memoize expensive computed values and useCallback() to memoize function references between renders.

12. How do you implement Server-Side Rendering (SSR)?

Use frameworks like Next.js (which handle SSR out of the box), or manually use ReactDOMServer.renderToString() on the server to generate initial HTML, then 'hydrate' it on the client with hydrateRoot() so React attaches event listeners to the existing markup.

13. How do you enable production mode in React?

When using a bundler (Webpack, Vite, etc.), setting NODE_ENV=production (or the equivalent build mode) triggers optimizations like stripping development warnings, enabling minification, and using the production build of the React package, which is significantly faster.

14. Do Hooks replace render props and higher-order components?

Hooks address most of the same use cases — sharing stateful logic between components — with less nesting and indirection ('wrapper hell'). While render props/HOCs are still valid patterns, custom hooks are now generally preferred for reusable logic.

15. What is a switching component?

A switching component renders one of several possible components based on some state or prop, similar to a switch statement — often implemented using conditional rendering or a mapping object of component types to render.

16. What are React Mixins?

Mixins were a legacy pattern (from React.createClass, pre-ES6 classes) for sharing reusable functionality across components by merging in extra methods/lifecycle logic. They're deprecated and discouraged today in favor of Hooks, HOCs, or composition.

17. What is the Virtual DOM and how does it work?

The Virtual DOM is an in-memory, lightweight representation of the real DOM. When state changes, React builds a new virtual DOM tree, diffs it against the previous one (reconciliation), and applies only the minimal set of real DOM updates needed, improving performance.

18. What is the difference between state and props?

props are read-only data passed down from a parent component to configure a child, and cannot be modified by the child. state is data managed internally within a component that can change over time and triggers re-renders when updated.

19. What are React hooks? Name a few common ones.

Hooks are functions that let function components use state and other React features without writing a class. Common ones include useState, useEffect, useContext, useRef, useMemo, useCallback, and useReducer.

20. What is the useEffect hook used for?

useEffect lets you perform side effects (data fetching, subscriptions, manual DOM changes) in function components, running after render, with an optional dependency array controlling when it re-runs, and an optional cleanup function for unmounting/preventing leaks.

21. What is the difference between useState and useReducer?

useState is best for simple, independent pieces of state. useReducer is better for complex state logic involving multiple sub-values or when the next state depends on the previous one, using a reducer function similar to Redux's pattern.

22. What is the Context API used for in React?

Context provides a way to pass data through the component tree without manually threading props at every level ('prop drilling'), useful for global data like themes, authenticated user info, or locale settings.

23. What is the difference between controlled and uncontrolled components?

A controlled component's form value is driven by React state (value + onChange), giving full control over input behavior. An uncontrolled component manages its own state internally in the DOM, accessed via refs when needed (e.g., on submit).

24. What is the key prop and why is it important in lists?

The key prop gives React a stable identity for list items across renders, allowing it to correctly track, reorder, add, or remove items during reconciliation rather than re-rendering the entire list.

Angular (20 questions)

1. What is the Angular Framework?

Angular is a TypeScript-based, opinionated front-end framework (by Google) for building single-page applications, providing built-in solutions for routing, forms, HTTP communication, dependency injection, and a component-based architecture.

2. What is the difference between AngularJS and Angular?

AngularJS (1.x) is based on JavaScript and uses a $scope/controller-based MVC architecture. Angular (2+) is a complete rewrite using TypeScript, a component-based architecture, better performance (via improved change detection), and mobile support.

3. What is TypeScript?

TypeScript is a statically-typed superset of JavaScript developed by Microsoft that compiles down to plain JavaScript, adding features like static types, interfaces, generics, and enhanced tooling/autocomplete support.

4. Write a pictorial diagram of Angular architecture.

At a high level: Modules (NgModules) organize the app; each module contains Components (view + logic), which use Templates (HTML), Directives, and Services (business logic/data), all wired together via Dependency Injection, with the Router managing navigation between views.

5. What are the key components of Angular?

Modules, Components, Templates, Directives, Services, Dependency Injection, Pipes, and the Router are the core building blocks that make up an Angular application.

6. What are directives?

Directives are classes that add behavior to elements in Angular templates. There are three types: Components (directives with a template), Structural directives (like *ngIf, *ngFor, which change DOM structure), and Attribute directives (like ngClass, ngStyle, which change appearance/behavior).

7. What are components?

A component is a directive with an associated template, controlling a patch of screen (a view). Each component consists of a TypeScript class, an HTML template, and optional CSS styles, decorated with @Component.

8. What are the differences between Component and Directive?

A component always has its own template/view and is a special type of directive. A regular directive (structural or attribute) has no template of its own — it only modifies the appearance or behavior of an existing DOM element.

9. What is a template?

A template is the HTML (with Angular-specific syntax like interpolation, directives, and bindings) that defines how a component's view is rendered.

10. What is a module?

An NgModule is a container that groups related components, directives, pipes, and services together, declaring what belongs to it and what it exports for use by other modules. The root module is typically called AppModule.

11. What are the lifecycle hooks available in Angular?

Key hooks include ngOnChanges, ngOnInit, ngDoCheck, ngAfterContentInit, ngAfterContentChecked, ngAfterViewInit, ngAfterViewChecked, and ngOnDestroy, called at specific points in a component's creation, update, and destruction.

12. What is data binding?

Data binding synchronizes data between a component class and its template. Angular supports interpolation ({{value}}), property binding ([property]), event binding ((event)), and two-way binding ([(ngModel)]).

13. What is metadata in Angular?

Metadata is configuration attached to a class via decorators (like @Component or @NgModule) that tells Angular how to process that class — e.g., its selector, template, and dependencies.

14. What is Angular CLI?

Angular CLI is a command-line tool for scaffolding, building, testing, and deploying Angular applications (ng new, ng generate, ng build, ng serve), automating much of the setup and tooling configuration.

15. What is the difference between constructor and ngOnInit?

The constructor is a standard TypeScript/JavaScript class feature used mainly for dependency injection, called when the class is instantiated (before Angular sets input properties). ngOnInit is an Angular lifecycle hook called once after Angular has initialized all data-bound input properties, making it the right place for initialization logic that depends on those inputs.

16. What is dependency injection in Angular?

Dependency Injection is a design pattern where a class receives its dependencies (like services) from an external source rather than creating them itself. Angular has a built-in DI system that supplies services to components/other services via constructor parameters.

17. What are Angular services?

Services are singleton (typically) classes decorated with @Injectable() that encapsulate reusable business logic, data access, or shared state, and are injected into components or other services instead of being duplicated in each component.

18. What is RxJS and why does Angular use it?

RxJS is a library for reactive programming using Observables, allowing composition of asynchronous and event-based operations (with operators like map, filter, switchMap). Angular uses it extensively for HTTP requests, forms, and the router for handling streams of async events.

19. What is the difference between Observable and Promise?

A Promise resolves once with a single value and cannot be cancelled. An Observable can emit multiple values over time, supports cancellation (unsubscribe), and provides a rich set of composable operators for transforming streams of data.

20. What is Angular routing and how does it work?

The Angular Router maps URL paths to components, enabling navigation within a single-page application without full page reloads. Routes are configured in a routing module (RouterModule.forRoot()), and <router-outlet> in a template marks where the matched component renders.